



INDIVIDUAL ASSIGNMENT - PART 1

ADVANCED SOFTWARE SECURITY

CT123-3-3-ASC

SECURE SOFTWARE PROPOSAL

BLOOD DONATION SYSTEM

INTAKE CODE: APD3F2601CS(CYB)

DATE ASSIGNED: 9TH JUNE 2026

DATE COMPLETED: 8TH AUGUST 2026

WEIGHTAGE: 40%

NAME: HEMANTHRAJAH A/L N KRISHNARAJAH

TP NUMBER: TP070844

Table of Contents

Abstract.....	4
1.0 Introduction.....	5
1.1 Target System and Purpose	5
1.2 Assessment Scope and Boundaries	7
1.3 Proposal Objectives	8
2.0 Proposed Secure Software Approach.....	9
2.1 Secure Design Method	9
2.2 Secure Coding Method	10
2.3 Secure Testing Method	11
2.4 Risk Prioritisation Method.....	11
3.0 Proposed Security Testing Tools	13
3.1 Semgrep	14
3.2 Bandit.....	15
3.3 OWASP ZAP	16
3.4 sqlmap	17
3.5 pip-audit.....	18
3.6 pytest	19
3.7 Nikto	20
3.8 Tool Selection Justification	21
4.0 Vulnerability Analysis and Proposed Solutions	22
4.1 Staff Login SQL Injection	23
4.2 Donor Search SQL Injection.....	24
4.3 Plaintext Staff Password Storage	25
4.4 Excessive Data Exposure and Missing Role Restriction	26

4.5 Predictable Donation Pass / Insecure Direct Object Reference.....	27
4.6 Stored Cross-Site Scripting.....	28
4.7 Missing Cross-Site Request Forgery Protection	28
4.8 Weak Session Security and Hardcoded Secret Key.....	29
4.9 Debug Mode and Verbose Error / SQL Query Leakage.....	29
5.0 Proposed Remediation and Verification Roadmap	32
5.1 Priority Plan	32
5.2 Proposed Secure Development Pipeline.....	32
5.3 Proposed Acceptance Criteria for Part 2.....	33
6.0 Conclusion	34
7.0 References.....	35
Appendix A Supplementary Proposal Evidence	38

Abstract

This is a proposal for a secure Software Analysis of the Blood Donation System deliberately developed using Flask, PostgreSQL, psycopg2 and Bootstrap. This application interfaces with donor identity information, health-screening data, donation history, blood-bank queue records, and medical-staff accounts; weaknesses in authentication, authorization, database interaction, browser output handling and session management can reveal personal and operational information. The proposal is based on assignment requirements that require analysis of the Secure Design, Secure Coding methods and Secure Testing. The assignment is to give reasons for the choice of appropriate tools before implementation and to justify them with respect to security. Assessment includes manual source-code review, Static application security testing (SAST), Dynamic application security testing (DAST), and targeted SQL injection validation, dependency auditing as well as web-server configuration checks. The preliminary scan of the vulnerable version revealed nine major weaknesses: SQL injection in staff login, SQL injection in donor search, insecure storage of passwords as clear text, poor data exposure for roles not restricted, predictable references to donation-pass, stored cross-site scripting, lack of CSRF protection, insecure session/configuration, and debug/verbose error disclosure. The proposed remediation strategy adheres to the defense in depth principle, and is not a single control such as parameterized database queries, secure password hashing, optional TOTP multi-factor authentication, server-side role-based authorization, random object references, output encoding, CSRF tokens, hardened session cookies, secure configuration and generic error handling. They are all tools that look at different layers of software security, so it makes sense to use the other tools as complementary. It makes sense to use Bandit, Semgrep, OWASP ZAP, sqlmap, pip-audit, and pytest and Nikto as complementary tools because each of them looks at a different layer of software security. This Part 1 report does not include any secure implementation evidence; implementation and post-remediation test results are included in Part 2.

1.0 Introduction

Applications on the web, which process identity, authentication and health information must be secured as a software quality attribute instead of a process conducted after software writing. The Blood Donation System employed in this assignment is intentionally vulnerable and hence serves as a controlled environment for an analysis of how insecure coding and design decisions impact on the confidentiality, integrity, authentication and operational trust. The current secure development guidance also calls for embedding security into the software development life cycle to allow vulnerabilities to be prevented, discovered and remediated systematically and not as a standalone defect (National Institute of Standards and Technology [NIST], 2022).

The proposed solution is structured around the three requirements assigned in the assignment Secure design, Secure coding and Secure testing. The design portion considers least privilege, defense in depth, secure defaults, data minimization and server-side authorization. During coding, the emphasis is on building queries using a database, authentication, encoding output, integrity of requests, handling sessions and safe failures. The testing part is a collection of independent tools since automated scanners cover different parts of the system and have differing false positives. Research on web-application testing has likewise found that combined scanning and manual validation provide broader coverage than reliance on one scanner alone (Abdulghaffar et al., 2023; Altulaihan et al., 2023).

The purpose of Part 1 is not to demonstrate the completed secure application. Instead, the vulnerable system is analysed, evidence is collected from the original environment, and a set of technically justified solutions is proposed. The actual secure coding changes, threat-model implementation details and post-remediation testing results will be documented separately in Part 2. This separation allows the proposal to establish a clear baseline and prevents the evidence of the vulnerable state from being confused with evidence of the later implementation.

1.1 Target System and Purpose

The target is a browser-based Blood Donation System implemented using Python, Flask, PostgreSQL, psycopg2 and Bootstrap. Two principal user groups are represented. Donors register personal information and may return for repeated donation activities, while medical staff authenticate to view donor records, perform health-screening updates, record donation status and

register collected blood in the blood-bank queue. The combination of personal identifiers, health-screening information, account credentials and operational records means that an authentication failure can rapidly become a wider confidentiality and authorization problem.

The original application is hosted only in a local laboratory environment owned and controlled by the tester. The assessment therefore remains within an authorised scope. NIST SP 800-115 describes security testing as a planned process for discovering vulnerabilities, analysing findings and developing mitigation strategies, which aligns well with this proposal because the goal is both discovery and remediation planning rather than uncontrolled exploitation (NIST, 2008).

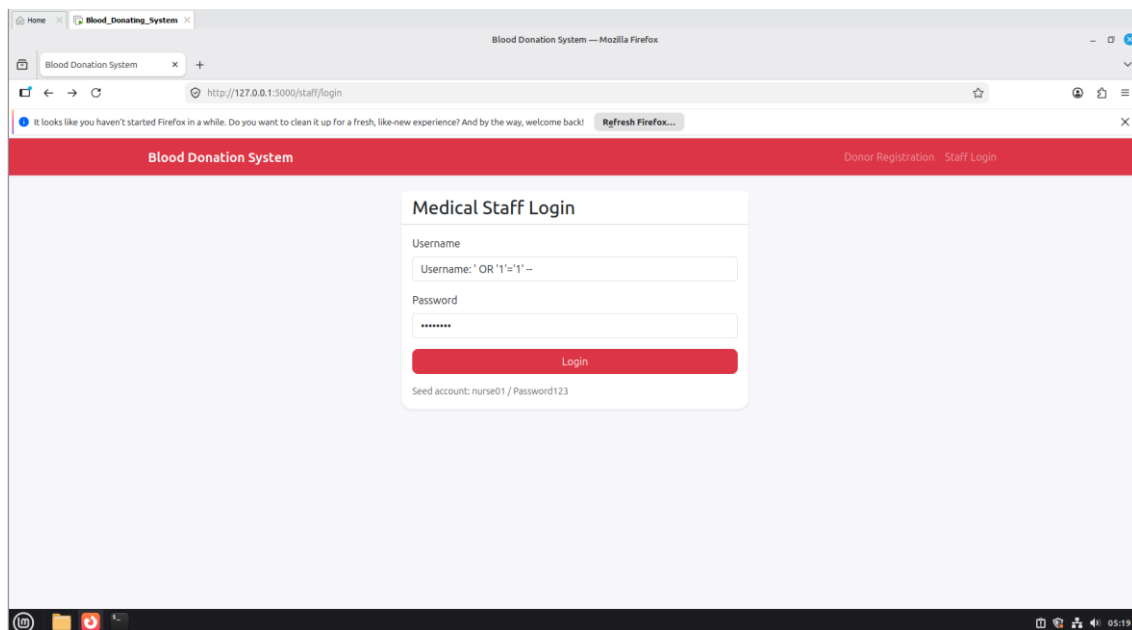


Figure 1: Original staff login interface with an SQL injection test payload

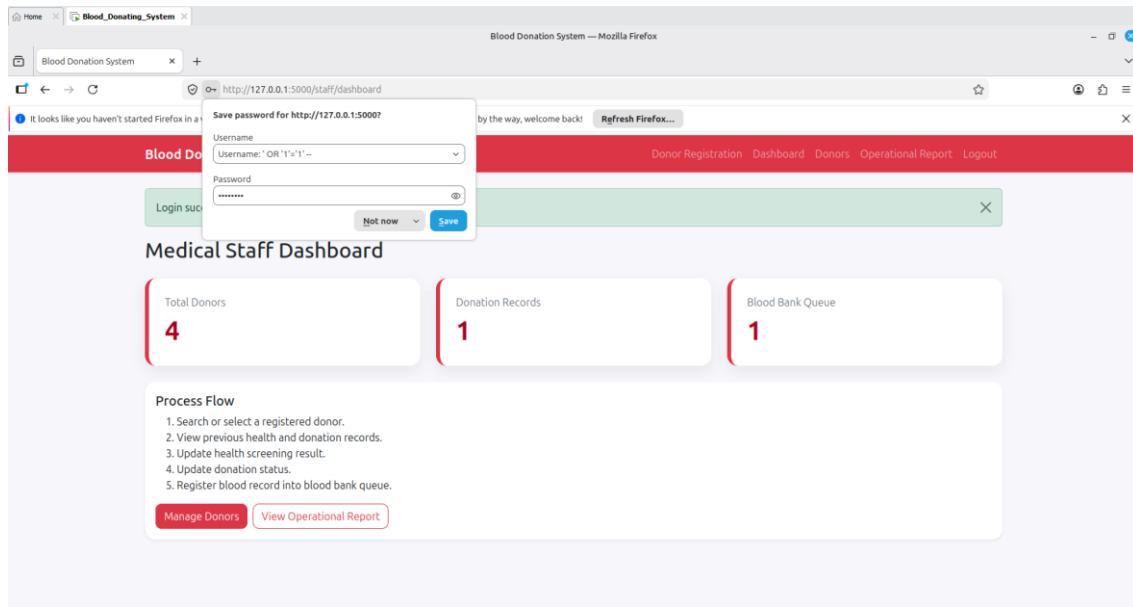


Figure 2: Original staff dashboard reached after vulnerable authentication

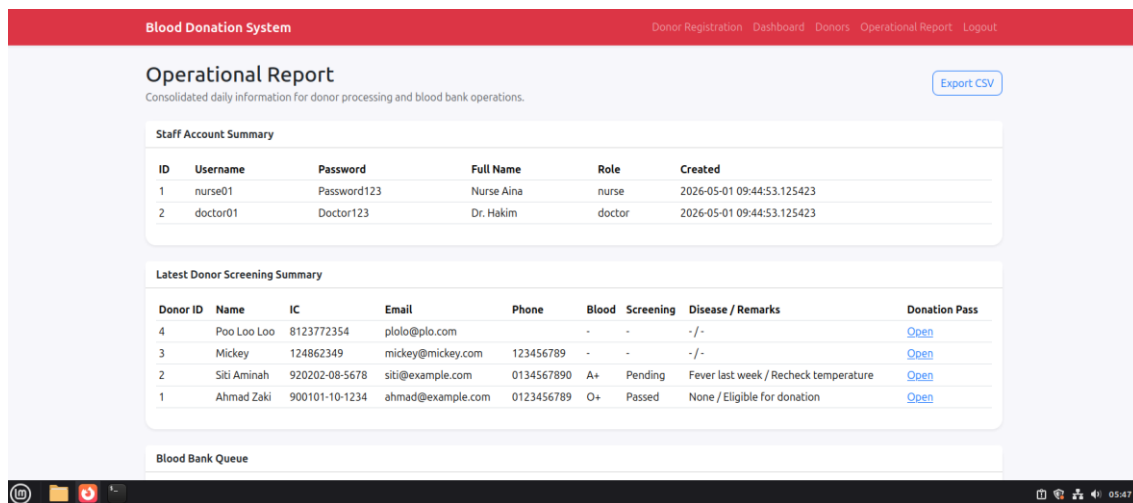


Figure 3: Original operational report containing consolidated staff and donor information

1.2 Assessment Scope and Boundaries

It is limited to the Blood Donation System running locally and its Flask application code, the interaction with PostgreSQL, its HTML templates, the Python dependencies and the HTTP behavior. The proposal will not involve denial-of-service testing, destructive change to the database, attacks on unrelated hosts or third-party systems or testing with actual patient or donor data. Data is gathered from synthetic or laboratory data. The testing strategy thus focuses on

scalable, nondestructive means of validation including query manipulation, inspection of HTTP requests, static code analysis and controlled configuration analysis.

Vulnerabilities to applications are clearly separated from general hardening opportunities. For instance, while sqlmap evidence can directly confirm SQL injections, Nikto may find a missing security header or exposed server information that do not by themselves make for an exploitable flaw. Similarly, a static analyzer can point out code patterns that must be reviewed by a human. Findings are therefore understood in context and are not necessarily true because a tool says so.

1.3 Proposal Objectives

- Using evidence from the source code analysis and controlled testing, identify and explain the key security weaknesses in the Blood Donation System.
- Suggest safe design controls to limit privilege, unnecessary exposure and reliance on one security measure.
- Recommend secure coding methods that directly resolve the root cause of vulnerabilities identified.
- Select and justify a complementary toolset for static analysis, dynamic testing, injection validation, dependency analysis, regression testing and web-server review.
- Define a remediation and verification plan that can be implemented and documented in Part 2 without mixing proposal evidence with implementation evidence.

2.0 Proposed Secure Software Approach

The proposed security approach is based on integrating secure requirements into the development workflow rather than applying isolated patches. NIST SSDF recommends practices that prepare the organisation, protect software, produce well-secured software and respond to vulnerabilities. For this application, those principles translate into clear security requirements for authentication, authorization, data handling, safe database access, secure configuration and repeatable testing (NIST, 2022). The OWASP Top 10:2025 is used as an application-risk reference because the identified issues map directly to areas such as Broken Access Control, Security Misconfiguration, Injection, Authentication Failures and Mishandling of Exceptional Conditions (OWASP Foundation, 2025a).

2.1 Secure Design Method

The proposed design method uses defence in depth. A critical control such as parameterized SQL must remove the root cause of injection, but additional controls should reduce the effect of a future defect. For the staff-login function, for example, the secure design should combine parameterized queries, strong password hashing, multi-factor authentication for staff, generic login messages, short-lived authenticated sessions and least-privilege database permissions. MFA is therefore not presented as an SQL injection fix; it is a second layer that protects accounts when passwords are compromised. OWASP recommends MFA broadly and specifically highlights TOTP as a practical option for web applications (OWASP Foundation, n.d.-g).

Authorization should be deny-by-default and enforced on the server. The system currently distinguishes roles such as nurse, doctor and potentially management functions, so sensitive operations should be granted only to roles that require them. An obvious example is the operational report. Authorization to view all personnel, donor, and queue data should not be implied by authentication alone. OWASP defines broken access control as allowing users to act outside intended permissions and recommends verifying authorization on every access attempt (OWASP Foundation, 2025b; OWASP Foundation, n.d.-f).

Data minimisation is another design requirement. Reports, exports and public-facing donation-pass pages should expose only information necessary for their business purpose.

Passwords, password hashes, MFA secrets and unnecessary personal identifiers should never appear in reports. Predictable identifiers should not be treated as authorization credentials. Complex random references can reduce enumeration, but OWASP stresses that random identifiers supplement rather than replace object-level authorization (OWASP Foundation, n.d.-f).

2.2 Secure Coding Method

Secure coding should first remove vulnerable data/directive mixing. The login and donor-search functions build SQL using user-controlled values inside formatted strings. OWASP recommends prepared or parameterized queries because the database then treats user input as data rather than SQL instructions. Input validation remains valuable as a secondary control but should not replace parameterization (OWASP Foundation, n.d.-a; MITRE, 2026a).

Credential handling should be redesigned so staff passwords are never stored or compared in plaintext. OWASP advises using adaptive password-hashing algorithms such as Argon2id or scrypt and explicitly states that passwords should not be stored in plain text (OWASP Foundation, n.d.-b). For a Flask implementation, a framework-supported password-hashing function can provide a practical student implementation while still maintaining one-way verification. A password policy, removal of default credentials, and MFA for privileged accounts should provide additional protection.

Browser-facing data should remain escaped by default. The current unsafe Markup filter intentionally bypasses normal Jinja escaping for stored health-screening fields. The secure version should remove unsafe rendering and rely on context-appropriate output encoding; content security policy can be added as defence in depth, but it should not replace correct output handling. OWASP describes output encoding and safe sinks as central XSS controls and warns against relying on a single secondary control such as CSP (OWASP Foundation, n.d.-c).

All state-changing POST operations should verify request authenticity using framework-supported CSRF tokens. OWASP recommends using built-in framework protection where available and validating CSRF tokens for state-changing requests. SameSite cookies and origin validation can supplement the token mechanism (OWASP Foundation, n.d.-d). Sessions should use an environment-provided secret, HttpOnly cookies, SameSite restrictions, Secure cookies

when HTTPS is used, appropriate timeouts and regeneration around authentication events (OWASP Foundation, n.d.-e).

Finally, the application should fail safely. Detailed SQL statements, stack traces and internal exception details should be recorded only in controlled server logs, while users receive generic responses. OWASP notes that verbose error handling can disclose technology and query information that helps reconnaissance and attack refinement (OWASP Foundation, n.d.-h). Debug mode should therefore be disabled by default outside a development-only environment.

2.3 Secure Testing Method

The testing method purposefully integrates multiple types of evidence. The SAST tool scans for unusual code patterns prior to execution, the DAST tool can be used to determine actual HTTP behavior, the targeted injection tooling can be used to test specific database flaws, the software composition analysis tool can be used to test third-party package vulnerabilities, the unit and regression tests can be used to validate whether security controls remain effective even after changes, and the web-server scanner can be used to test configuration-level issues. This multi-layered approach aligns with studies demonstrating that several scanners can enhance the vulnerability coverage as each scanner has unique detection capabilities and false-positive behavior (Abdulghaffar et al., 2023; Altulaihan et al., 2023).

Manual validation is still required because there may be weaknesses in the business logic, such as the ability to receive donations by “passing” the report or giving too many report privileges, that a generic scanner won't detect. The testing plan then considers the output of the scanner as evidence to be interpreted. The same test cases used against the vulnerable baseline should be applied to the secure baseline to show that control affects the observable baseline behaviors instead of just changing the appearance of the baseline code.

2.4 Risk Prioritisation Method

The proposed classification of the severity of the impacts on confidentiality, integrity and authentication of the affected data is based on the qualitative criterion of severity which is defined according to the level of exploitability, the level of privileges needed for the attack, the sensitivity of the affected data and the potential impact on the three security criteria. The staff-login SQL

injection is rated Critical as it has been shown to bypass authentication and could give access to additional application functionality. Donor-search injection, plaintext credentials, and too much report exposure and predictable donation-pass access are considered High, as they may allow the exposure of sensitive information or privilege misuse. For this lab context, XSS, CSRF, weak sessions and debug/error leakage are considered a medium threat, but can become a high threat if chained with more severe vulnerabilities.

3.0 Proposed Security Testing Tools

The number of seven tools is chosen because of the fact that they cover various layers of the target. Semgrep scans source code, Bandit checks for server and HTTP configuration issues, sqlmap validates SQL injection, pip-audit checks for known dependency vulnerabilities, pytest enables repeatable regression testing, and Nikto checks server and HTTP configuration. It's not about how many tools are better to have, each of these tools addresses a different question that the proposal needed to ask.

Tool	Category	Primary Proposal Role	Main Strength	Important Limitation
Semgrep	SAST	Broad source-code rule scanning	Fast multi-rule code analysis and clear finding locations	Rules can produce false positives and require review
Bandit	Python SAST	Python-specific security patterns	AST-based checks tailored to Python	Does not understand all application business logic
OWASP ZAP	DAST / proxy	HTTP inspection and web security validation	Captures requests/responses and supports passive/active analysis	Authenticated workflow coverage needs careful configuration
sqlmap	Targeted injection testing	Confirm SQL injection in GET/POST parameters	Specialised DBMS-aware injection detection	Narrow scope; should not replace general testing
pip-audit	SCA	Known vulnerabilities in Python dependencies	Audits requirements and provides fix versions	Only detects published package advisories
pytest	Regression testing	Automate expected security behaviours after remediation	Readable, repeatable tests integrated with Python project	Tests only what has been explicitly written
Nikto	Web-server scanner	Headers, server disclosure and web configuration	Fast broad checks of web-server exposure	Many results are informational and need context

Table 1: Tool-to-activity comparison for the proposed assessment

3.1 Semgrep



Figure 4: Semgrep logo

Semgrep is chosen as the wide SAST module. It is documented to have the capability of locally scanning a project with security rules and attaching findings to source locations (Semgrep, n.d.). This helps the Blood Donation System, since the most severe issues come from the construction of the code; formatted SQL strings, unsafe output handling, hard-coded configuration and debug behaviors. The vulnerable baseline scan resulted in 64 baseline violations in the target set, successfully scanned, showing the value of static analysis for identifying patterns for manual review. The number does not mean that 64 vulnerabilities were confirmed, it is a triage result and needs to be matched with source code and runtime evidence.

```
Scan Summary
✓ Scan completed successfully.
• Findings: 64 (64 blocking)
• Rules run: 237
• Targets scanned: 1503
• Parsed lines: ~100.0%
• Scan skipped:
  ◦ Files without read access: 5
  ◦ Files larger than files 1.0 MB: 3
  ◦ Files matching .semgrepignore patterns: 7
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 237 rules on 1503 files: 64 findings.

🔊 Too many findings? Try Semgrep Pro for more powerful queries and less noise.
See https://sg.run/false-positives.

niks@Hemanth-virtual-machine:/opt/blood-donation-system$
```

Figure 5: Semgrep scan summary for the original vulnerable system

3.2 Bandit



Figure 6: Bandit logo

Bandit improves on Semgrep by focusing specifically on common Python security issues. Bandit constructs an abstract syntax tree for Python code and adds security plugins to the appropriate nodes, so it is suitable for a Flask application (PyCQA, n.d.-a). The initial application scan found one High and four Low problems. The benefit of this approach is that it is Python-specific, the drawback is that a static pattern is not necessarily a verified attack, and the scanner doesn't necessarily know the intended authorization model of the application.

```
Code scanned:
  Total lines of code: 323
  Total lines skipped (#nosec): 0

Run metrics:
  Total issues (by severity):
    Undefined: 0.0
    Low: 4.0
    Medium: 0.0
    High: 1.0
  Total issues (by confidence):
    Undefined: 0.0
    Low: 0.0
    Medium: 1.0
    High: 4.0
Files skipped (0):
niks@hemant-virtual-machine:/opt/blood-donation-system$ bandit -r . -x ./venv,./.venv,./evidence,./__pycache__ -f txt -o evidence/bandit_old.txt
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.12.3
[text] INFO Text output written to file: evidence/bandit_old.txt
```

Figure 7: Bandit summary for the original vulnerable Python application

3.3 OWASP ZAP



Figure 8: OWASP ZAP logo

The proposed primary HTTP interception and DAST platform is OWASP ZAP. ZAP offers an intercepting proxy, passive and active scanners, spiders and visibility of request/response. It is a passive scanner that does not alter proxied traffic and is well suited for problems like missing security headers and anti-CSRF indicators (ZAP by Checkmarx, n.d.). For the purposes of this project, ZAP is useful because it captures all logins and donor searches made while under test and provides the corresponding HTTP responses as evidence.

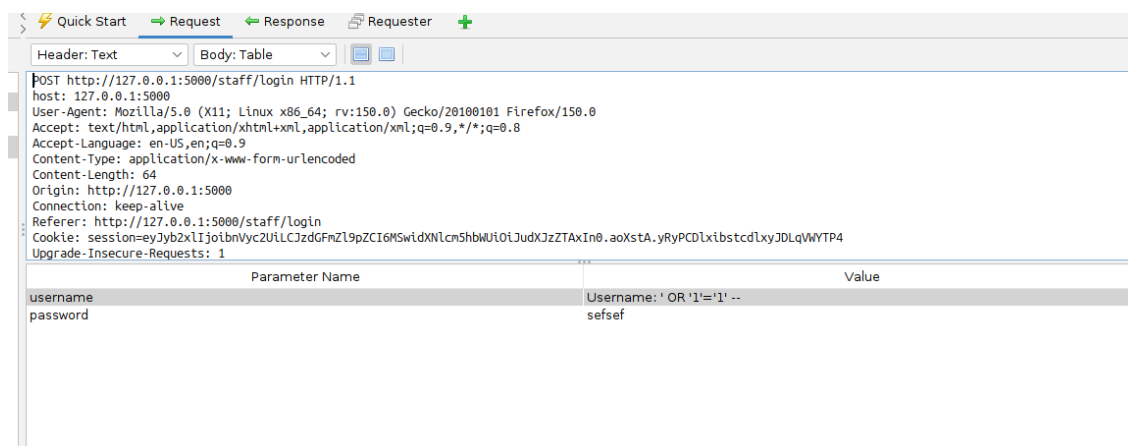


Figure 9: ZAP capture of the vulnerable staff-login request

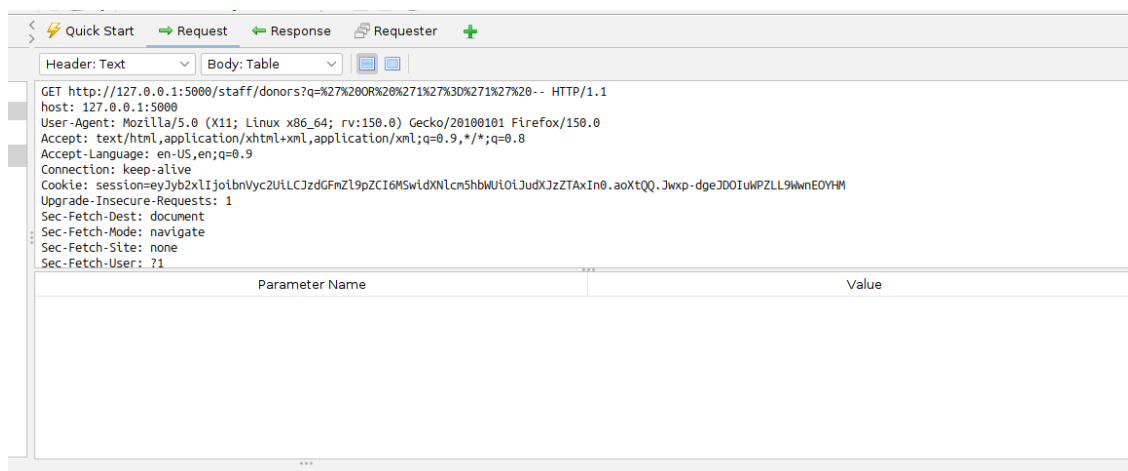


Figure 10: ZAP capture of the vulnerable donor-search request

3.4 sqlmap

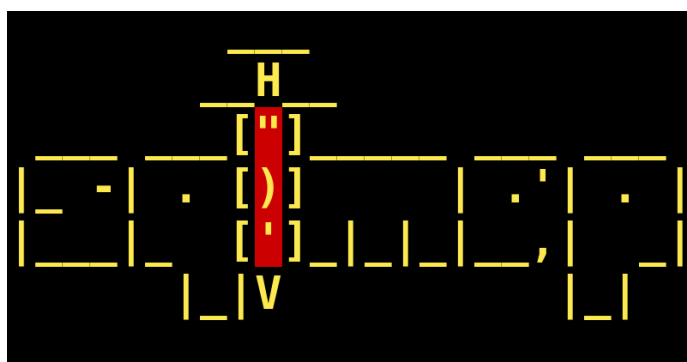


Figure 11: sqlmap logo

sqlmap has been designed as a special validator and not as a scanning tool. It has documentation for usage that enables you to test GET/POST parameters and can be used for the selection of individual parameters to be injected for detection (sqlmap project, 2026). This exactly matches the two injection points of the application: the staff login form and donor search query parameter. In the system that was vulnerable, sqlmap was able to find several injection points involving the login input and also found that the input parameter used to search for donors was injectable. This targeted confirmation differs from just a static warning as the assignment will be accompanied by evidence that must be justified.

```

---
Parameter: #1* ((custom) POST)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause (subquery - comment)
Payload: username= AND 9532=(SELECT (CASE WHEN (9532=9532) THEN 9532 ELSE (SELECT 6640 UNION SELECT 7726) END))-- mR2ndpassword=anything

Type: error-based
Title: PostgreSQL AND error-based - WHERE or HAVING clause
Payload: username= AND 6173=CAST((CHR(113))||(CHR(107))||(CHR(122))||(CHR(112))||(CHR(113)))|(SELECT (CASE WHEN (6173=6173) THEN 1 ELSE 0 END))::text||(CHR(113))||(CHR(113))||(CHR(98))||(CHR(120))||(CHR(113))) AS NUMERIC) AND
hyVd '=hyVd6password=anything

Type: stacked queries
Title: PostgreSQL > 8.1 stacked queries (comment)
Payload: username=';SELECT PG_SLEEP(5)--$password=anything

Type: time-based blind
Title: PostgreSQL > 8.1 AND time-based blind
Payload: username= AND 2224=(SELECT 2224 FROM PG_SLEEP(5)) AND 'ZfQ6'='ZfQ66password=anything

---
[10:56:07] [INFO] the back-end DBMS is PostgreSQL
back-end DBMS: PostgreSQL
[10:56:07] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 89 times
[10:56:07] [INFO] fetched data logged to text files under '/home/niks/.local/share/sqlmap/output/127.0.0.1'
[10:56:07] [WARNING] your sqlmap version is outdated

```

Figure 12: sqlmap confirmation of SQL injection in the original staff-login request

```

que test
[11:07:59] [INFO] target URL appears to have 9 columns in query
do you want to (re)try to find proper UNION column types with fuzzy test? [y/N] N
injection not exploitable with NULL values. Do you want to try with a random integer value for option '--union-char'? [Y/n] Y
[11:08:02] [INFO] testing 'Generic UNION query (NULL) - 21 to 40 columns'
[11:08:02] [INFO] checking if the injection point on GET parameter 'q' is a false positive
GET parameter 'q' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 168 HTTP(s) requests:
---
Parameter: q (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause (subquery - comment)
Payload: q=test' AND 3269=(SELECT (CASE WHEN (3269=3269) THEN 3269 ELSE (SELECT 5122 UNION SELECT 4474) END))-- ALGY

Type: stacked queries
Title: PostgreSQL > 8.1 stacked queries (comment)
Payload: q=test';SELECT PG_SLEEP(5)--

---
[11:08:03] [INFO] the back-end DBMS is PostgreSQL
back-end DBMS: PostgreSQL
[11:08:03] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 145 times
[11:08:03] [INFO] fetched data logged to text files under '/home/niks/.local/share/sqlmap/output/127.0.0.1'
[11:08:03] [WARNING] your sqlmap version is outdated

[*] ending @ 11:08:03 /2026-08-20/

```

Figure 13: sqlmap confirmation of SQL injection in the donor-search parameter

3.5 pip-audit



Figure 14: Python packaging logo used for the pip-audit tool section

pip-audit is suggested for the software composition analysis. The tool scans Python environments, requirements files and dependency trees for known vulnerabilities, and can also determine if a fixed version was published for a vulnerability if the version is available (Python

Packaging Authority, n.d.). The initial requirements scan identified 9 known vulnerabilities in 4 packages: Flask, python-dotenv, requests and Werkzeug. It is a concern in regards to software supply-chain failures as one of the Top 10 OWASP 2025 risks defined: "Track versions and apply updates as per (a risk-based) approach" (OWASP Foundation, 2025d).

```

nicks@Hemanth-virtual-machine:/opt/blood-donation-system$ pip-audit -r requirements.txt
Found 9 known vulnerabilities in 4 packages
Name          Version ID      Fix Versions
-----
flask          3.0.3           PYSEC-2026-2151 3.1.3
python-dotenv 1.0.1           PYSEC-2026-2270 1.2.2
requests       2.32.3          PYSEC-2026-1872 2.32.4
requests       2.32.3          PYSEC-2026-2275 2.33.0
werkzeug       3.0.3           PYSEC-2026-2045 3.0.6
werkzeug       3.0.3           PYSEC-2026-2046 3.1.4
werkzeug       3.0.3           PYSEC-2026-2044 3.1.5
werkzeug       3.0.3           PYSEC-2026-2320 3.1.6
werkzeug       3.0.3           PYSEC-2026-3417 3.0.6
nicks@Hemanth-virtual-machine:/opt/blood-donation-system$

```

Figure 15: pip-audit findings against the original system requirements

3.6 pytest



Figure 16: pytest logo

pytest is proposed for the post-remediation regression layer. Unlike a vulnerability scanner, pytest does not discover unknown security flaws automatically; its value is that expected security behaviour can be encoded as repeatable tests. The official documentation describes pytest as a framework for small, readable tests that can scale to complex functional testing (pytest developers, n.d.). In Part 2, tests should verify that SQL injection payloads do not change query structure, CSRF-free POST requests are rejected, role restrictions are enforced, predictable donation-pass references fail, XSS is escaped and MFA/session flows behave as designed. Implementation test screenshots are intentionally omitted from this proposal.

3.7 Nikto



Figure 17: Nikto logo

Nikto is selected as a supporting web-server scanner. Its manual describes checks for server/software misconfiguration, default or insecure files, outdated software and other web-server problems (Sullo & Lodge, 2026). The vulnerable baseline identified the Werkzeug/Python server banner, absence of the X-Frame-Options anti-clickjacking header and an interesting /console path. These findings do not replace application-level testing, but they illustrate that safe software review must also address deployment settings and HTTP response hardening.

```
niks@Hemanth-virtual-machine:~$ nikto -h http://127.0.0.1:5000
- Nikto v2.1.5
-----
+ Target IP:      127.0.0.1
+ Target Hostname: localhost
+ Target Port:    5000
+ Start Time:     2026-08-20 11:45:27 (GMT-4)
-----
+ Server: Werkzeug/3.0.3 Python/3.12.3
+ The anti-clickjacking X-Frame-Options header is not present.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Allowed HTTP Methods: OPTIONS, HEAD, GET
+ OSVDB-3092: /console: This might be interesting...
+ 6544 items checked: 0 error(s) and 3 item(s) reported on remote host
+ End Time:      2026-08-20 11:45:42 (GMT-4) (15 seconds)
-----
+ 1 host(s) tested
niks@Hemanth-virtual-machine:~$
```

Figure 18: Nikto findings for the original vulnerable web application

3.8 Tool Selection Justification

The set of tools has been chosen with a deliberate overlap at significant boundaries. Both Semgrep and Bandit can warn on issues in source code, and Semgrep has a set of more general configurable rules, with Bandit focusing on Python. ZAP and sqlmap both inject what is deemed general HTTP context, whereas pip-audit concentrates on a different supply-chain layer, pytest offers repeatability following code changes and Nikto injects deployment-level observations. Using both helps to minimize reliance on a single scanner and meets the need for detailed justification of using secure testing tools in the assignment. Tools are thus indicated as supporting evidence sources rather than being a replacement for secure design or human analysis in the proposal.

4.0 Vulnerability Analysis and Proposed Solutions

For the reasons outlined above, nine principal vulnerabilities were identified for the proposal, including database security, authentication, authorization, browser security, request integrity, session management and secure configuration. The solutions below are proposals only. This report does not include any fixed-system screenshots or final implementation claims.

No.	Vulnerability	Severity	Primary OWASP/CWE Alignment	Primary Proposed Fix
1	Staff login SQL injection	Critical	OWASP A05:2025 / CWE-89	Parameterized query; hash verification outside SQL
2	Donor search SQL injection	High	OWASP A05:2025 / CWE-89	Parameterized ILIKE query and validation
3	Plaintext staff password storage	High	OWASP A04/A07:2025	Adaptive password hashing; remove plaintext
4	Excessive data exposure / weak role restriction	High	OWASP A01:2025	Server-side RBAC and data minimisation
5	Predictable donation-pass reference / IDOR	High	OWASP A01:2025	Object authorization + random token
6	Stored cross-site scripting	Medium	OWASP A05:2025 / CWE-79	Output encoding; remove unsafe rendering
7	Missing CSRF protection	Medium	OWASP A01:2025 / CWE-352	Framework CSRF tokens on state changes
8	Weak session security / hardcoded secret	Medium	OWASP A02/A07:2025	Environment secret; secure cookie settings
9	Debug mode and verbose error leakage	Medium	OWASP A02/A10:2025	Debug off; generic errors; server logging

Table 2: Summary of identified vulnerabilities and proposed primary controls

4.1 Staff Login SQL Injection

The most serious is the staff-login one as it directly forms the authentication SQL statement with user-controlled username and password. In the original code, a Python formatted string is used to build the SQL query and then the string is executed. In the original code, the SQL query is built using a Python formatted string and then the query is executed. CWE-89: Externally influenced input is incorporated into an SQL command and can thus be interpreted as SQL syntax instead of regular input data (MITRE, 2026a).

```
query = f"SELECT id, username, full_name, role FROM medical_staff WHERE username = '{username}' AND password = '{password}'"
cur.execute(query)
```

```
@app.route("/staff/login", methods=["GET", "POST"])
def staff_login():
    if request.method == "POST":
        username = request.form.get("username", "")
        password = request.form.get("password", "")

        # INTENTIONAL CRITICAL VULNERABILITY: SQL Injection in login query.
        query = f"SELECT id, username, full_name, role FROM medical_staff WHERE username = '{username}' AND password = '{password}'"

        conn = get_conn()
        cur = conn.cursor()
        try:
            cur.execute(query)
            staff = cur.fetchone()
        except Exception as e:
            # INTENTIONAL LOW VULNERABILITY: verbose error leakage.
            return render_template("error.html", error=str(e), query=query), 500
        finally:
            cur.close()
            conn.close()
```

Figure 19: Vulnerable staff-login SQL construction in the original application

The weakness was actively tested. The malicious login box was submitted to the vulnerable application, which allowed staff view access to the staff dashboard. ZAP captured that traffic and was able to capture the redirect traffic, while sqlmap found Boolean-based, error-based, stacked-query, and time-based methods to the POST parameter. This is a mixture of source and runtime evidence that supports a Critical rating as the flaw is not simply a runtime error, but an error that breaches the authentication boundary.

Parameter Name	Value
username	Username: ' OR '1'='1' --
password	selfef

ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note	Tags
1	Proxy	8/19/26, 1:51:06 PM	GET	http://127.0.0.1:5000/staff/dashboard	200	OK	31 ms	2,488 bytes	Medium	Script	
3	Proxy	8/19/26, 1:51:06 PM	GET	http://127.0.0.1:5000/static/css/style.css	304	NOT MODIFIED	5 ms	0 bytes			
7	Proxy	8/19/26, 1:51:57 PM	GET	http://127.0.0.1:5000/staff/login	200	OK	14 ms	2,231 bytes	Medium	Form, Password, Script	
8	Proxy	8/19/26, 1:51:57 PM	GET	http://127.0.0.1:5000/static/css/style.css	304	NOT MODIFIED	10 ms	0 bytes			
9	Proxy	8/19/26, 1:52:00 PM	POST	http://127.0.0.1:5000/staff/login	302	FOUND	55 ms	219 bytes	Low	SetCookie	
10	Proxy	8/19/26, 1:52:01 PM	GET	http://127.0.0.1:5000/staff/dashboard	200	OK	138 ms	2,731 bytes			
11	Proxy	8/19/26, 1:52:01 PM	GET	http://127.0.0.1:5000/static/css/style.css	304	NOT MODIFIED	7 ms	0 bytes			

Figure 20: Runtime evidence of authentication bypass in the vulnerable version

The main suggested remedy is to eliminate the use of dynamic SQL construction altogether. Implementation phase should involve getting the account with a parameterized username query and checking the provided password against a stored password hash in application code. The advantage of having prepared/parameterized queries is that the structure of the query is separated from the user submitted data, making it the recommended approach to SQL injection defense (OWASP Foundation, n.d.-a). Other controls should include least-privilege database permissions, login throttling, generic authentication failures, and secure session creation and staff MFA with TOTP. MFA is not an SQL injection solution, but rather it is an account compromise mitigating solution (OWASP Foundation, n.d.-g; NIST, 2025).

4.2 Donor Search SQL Injection

There is another SQL injection vulnerability on the donor search route. The string you want to match is contained in an ILIKE expression that is passed in the query parameter q on the request. While this route does require a staff session, once it is done, it can reveal or enumerate donor records and also aid in subsequent attacks, in contrast to the login vulnerability. The presence of two injection points also shows why fixing only the login query would not constitute a complete remediation.

```
keyword = request.args.get('q', "")
query = f"SELECT * FROM donors WHERE full_name ILIKE '%{keyword}%' OR ic_number ILIKE '%{keyword}%'
ORDER BY id DESC"
cur.execute(query)
```

```
@app.route("/staff/donors")
def staff_donors():
    if not staff_required():
        return redirect(url_for("staff_login"))

    keyword = request.args.get("q", "")
    conn = get_conn()
    cur = conn.cursor()

    # INTENTIONAL HIGH/MEDIUM ISSUE: SQL Injection in donor search.
    if keyword:
        query = f"SELECT * FROM donors WHERE full_name ILIKE '%{keyword}%' OR ic_number ILIKE '%{keyword}%' ORDER BY id DESC"
    else:
        query = "SELECT * FROM donors ORDER BY id DESC"

    try:
        cur.execute(query)
        donors = cur.fetchall()
    except Exception as e:
        return render_template("error.html", error=str(e), query=query), 500
    finally:
        cur.close()
        conn.close()

    return render_template("staff_donors.html", donors=donors, keyword=keyword)
```

Figure 21: Vulnerable donor-search SQL construction in the original application

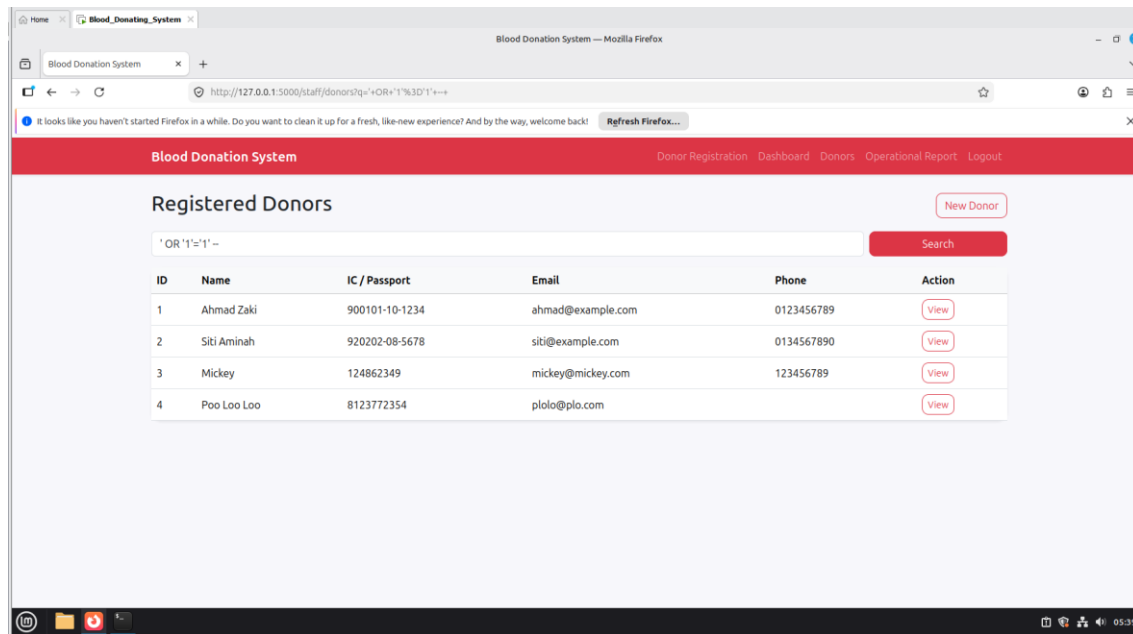


Figure 22: Donor search manipulated to return records in the vulnerable application

The proposed fix is a parameterized search statement in which the wildcard pattern is supplied as a parameter rather than inserted into the SQL text. The search input should additionally have a reasonable length limit and type/content validation to improve robustness, but validation is secondary to parameterization. Database privileges should be limited so that even a future query defect cannot automatically access unrelated tables. The post-remediation acceptance test should repeat the same q payload through ZAP/sqlmap and verify that the input is treated literally and no SQL error or broadened result set is produced (OWASP Foundation, n.d.-a).

4.3 Plaintext Staff Password Storage

The original login query does a direct comparison of the password against the `medical_staff.password` field; the operational report specifically selects and displays the column named `password`. The behaviors indicate that staff credentials are not stored as one-way hashes, but in recoverable plaintext. As a result, the disclosure of a database or report provides an attacker with usable credentials at the moment and may facilitate reuse of those credentials on other systems if employees reuse their passwords.

```
SELECT id, username, password, full_name, role, created_at
FROM medical_staff ORDER BY id
```

The suggested solution is to switch from the plaintext storage to an adaptive password-hashing scheme. OWASP recommends Argon2id and, where it is unavailable, scrypt; it explicitly advises that passwords should never be stored in plaintext (OWASP Foundation, n.d.-b). For the implementation phase, the schema should retain only password_hash, seed/demo accounts should be generated as hashes, and login should use a framework-supported hash-verification function. Password or hash fields must not be included in operational reports or CSV exports. MFA, password-quality controls, removal of default credentials and controlled password-reset procedures should provide additional protection.

4.4 Excessive Data Exposure and Missing Role Restriction

The operational report is available to any authenticated staff account in the original design and consolidates staff account information, donor personal identifiers, screening results and blood-bank queue records. It also supports CSV export. The problem is therefore broader than confidentiality of one field: the route creates a high-value aggregation point and does not distinguish between ordinary clinical users and users who genuinely require organisation-wide reporting. OWASP A01:2025 describes access-control failures as allowing users to perform actions or obtain information outside their intended permissions (OWASP Foundation, 2025b).

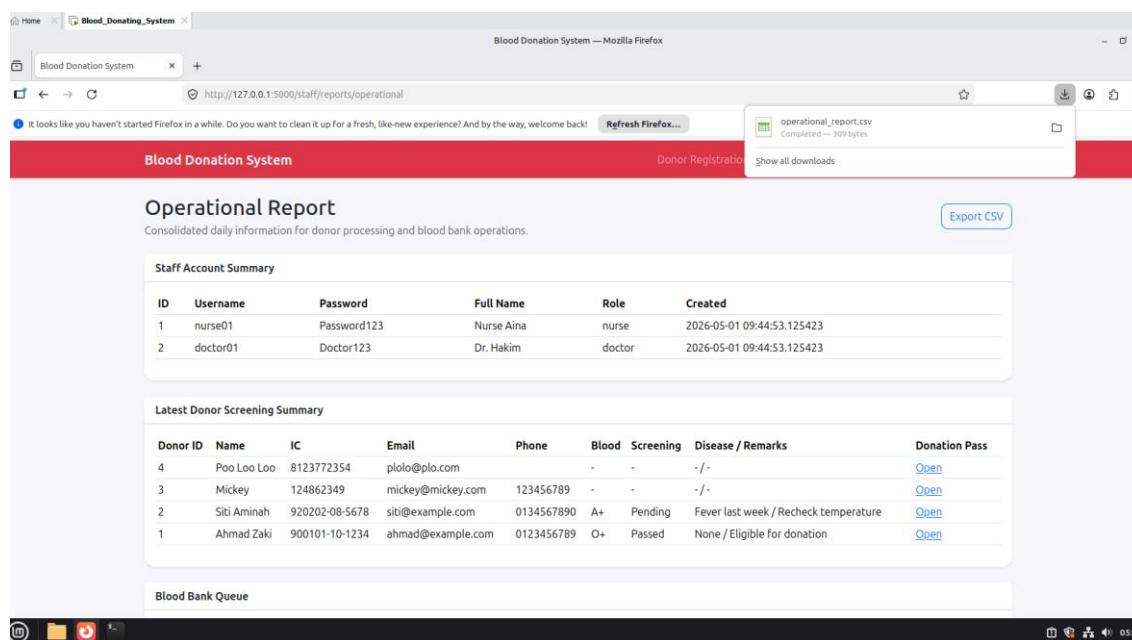


Figure 23: Original operational report/export capability exposing consolidated information

The proposed design should implement server-side role-based access control with deny-by-default behaviour. A manager/admin role should be required for the operational report and export; hiding a navigation button alone is insufficient because authorization must also protect direct URL requests. The report query should remove all password/authentication fields and minimise donor PII. Where full identifiers are unnecessary, values should be masked or omitted. Report views and exports should also be logged as security-sensitive actions. This implements least privilege and data minimisation and reduces the blast radius of a compromised lower-privilege account.

4.5 Predictable Donation Pass / Insecure Direct Object Reference

The donation-pass feature uses a reference derived directly from the donor identifier, following the form BDP-{donor_id}. The route does not require authentication and checks whether the supplied reference equals the predictable value. Once an attacker learns or guesses a donor identifier, the matching pass reference can be constructed without possession of a secret. The page then returns donor information, the latest screening record and donation history. This is consistent with IDOR: access to an object is controlled by a manipulable reference without sufficient object-level authorization (OWASP Foundation, n.d.-f).

```
donor_id = request.args.get('donor_id', "")
ref = request.args.get('ref', "")
expected_ref = f'BDP-{donor_id}'
if not donor_id or ref != expected_ref:
    ...
```

The proposed solution has two layers. First, server-side authorization should determine whether the requester is allowed to access the pass. Where an unauthenticated donor pass is required by the business workflow, the public reference should be a long cryptographically random token stored server-side instead of a value derived from a sequential donor ID. Second, the page should expose only the minimum information required for the pass purpose. Optional expiry, token rotation/revocation and access logging further reduce risk. OWASP notes that complex random identifiers are useful defence in depth but should not replace access-control checks (OWASP Foundation, n.d.-f).

4.6 Stored Cross-Site Scripting

Health-screening disease and remarks fields are stored from staff input, and the original application defines an unsafe template filter that returns Markup directly. This bypasses Jinja's normal escaping and creates a stored XSS pathway if script-capable content is saved and later rendered. Stored XSS is especially important in this system because the eventual viewer is likely to be authenticated medical staff, so malicious script executes in a privileged browser context. CWE-79 describes the weakness as failure to neutralize user-controllable input before it is placed in web output (MITRE, 2026b).

```
@app.template_filter('unsafe')
def unsafe_filter(value):
    return Markup(value or "")
```

The proposed fix is to remove unsafe rendering for all user-controlled fields and rely on Jinja auto-escaping/context-appropriate output encoding. Input validation and field length constraints should prevent malformed or excessive data but should not be treated as the primary XSS defence. A Content Security Policy can be proposed as an additional browser layer, while HttpOnly session cookies reduce the value of cookie-stealing scripts. OWASP recommends framework escaping, output encoding and safe sinks as the central controls for XSS prevention (OWASP Foundation, n.d.-c).

4.7 Missing Cross-Site Request Forgery Protection

The original application includes multiple authenticated POST routes for health screening, donation status and blood-record creation, but the Flask application does not initialise a CSRF protection mechanism and the forms do not require unpredictable anti-CSRF tokens. A malicious site could therefore cause a logged-in staff browser to send an unwanted state-changing request while the session cookie is automatically attached.

The proposed implementation should use Flask-WTF or an equivalent framework-supported CSRF mechanism and place a validated token on every state-changing form, including authentication and MFA-management forms. OWASP recommends framework-provided CSRF protection where available and token validation for state-changing requests. SameSite cookie settings, Origin/Referer validation and re-authentication for especially sensitive operations provide

additional layers (OWASP Foundation, n.d.-d). The Part 2 test should demonstrate that a valid form succeeds while the same POST without a token is rejected.

4.8 Weak Session Security and Hardcoded Secret Key

The original Flask configuration uses a predictable hardcoded secret and explicitly disables both HttpOnly and Secure cookie settings. The secret is important because Flask signs session data using the application secret; hardcoding it in source creates unnecessary exposure through source repositories, backups or code sharing. Disabling HttpOnly increases the potential effect of XSS by allowing JavaScript to access the cookie, while disabling Secure permits the cookie to be transmitted over unencrypted HTTP when deployed that way.

```
app.secret_key = 'blood-donation-lab-secret'
app.config['SESSION_COOKIE_HTTPONLY'] = False
app.config['SESSION_COOKIE_SECURE'] = False
```

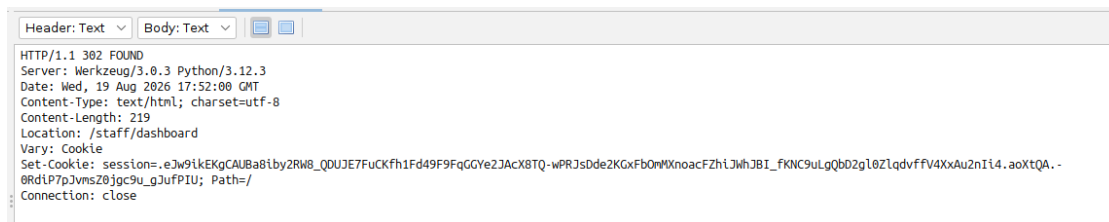


Figure 24: ZAP evidence of the original session-cookie configuration

The proposed solution is to store the application secret in a protected environment variable and to store the high entropy generated value. HttpOnly and SameSite should be enabled for session cookies, and Secure should be enabled on the cookies when the application is served over HTTPS. It should also implement reasonable session inactivity/absolute timeout values and fully invalidate the session when it is logged out and re-issue it during login/MFA transitions. OWASP states that the session token from a user is treated as a user's authentication as long as it is valid, and therefore needs to be protected (OWASP Foundation, n.d.-e).

4.9 Debug Mode and Verbose Error / SQL Query Leakage

The original application is loaded with Flask running in debug mode, and in the event of a SQL exception, it returns to error.html the exception text and the constructed SQL query. This creates two related security problems: an unsafe runtime configuration and information disclosure during exceptional conditions. The donor-search route can be forced into a database syntax error,

after which the user receives internal database information that can help refine later injection attempts. OWASP recommends returning generic errors to users and recording detailed diagnostic information only in controlled server-side logs (OWASP Foundation, n.d.-h).

```
except Exception as e:
    return render_template('error.html', error=str(e), query=query), 500
...
app.run(host='0.0.0.0', port=5000, debug=True)
```



Figure 25: Verbose SQL query/error disclosure in the original application

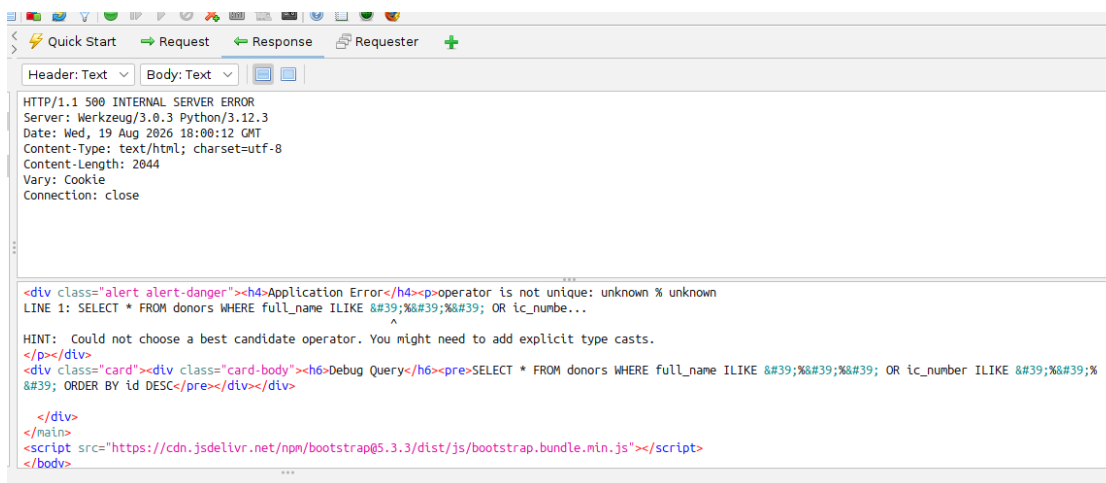


Figure 26: ZAP response showing internal database error information

The proposed fix is to disable debug mode by default and make any development-only debug setting explicit through the environment. User-facing errors should contain a generic message and a suitable HTTP status while detailed exceptions are logged on the server without passwords, MFA secrets, session tokens or full SQL statements. A further hardening step is to add response headers such as X-Frame-Options or an equivalent CSP frame-ancestors policy, X-Content-Type-Options and Referrer-Policy. The vulnerable Nikto scan already showed that the

anti-clickjacking header was missing, making this a useful deployment acceptance criterion for Part 2.

5.0 Proposed Remediation and Verification Roadmap

The vulnerabilities should not be remediated in an arbitrary order. The first implementation stage should remove paths that permit authentication bypass or direct credential disclosure. The second stage should enforce authorization and browser/request integrity. The third stage should harden configuration, dependencies and repeatable testing. This order is based on severity as well as dependency: In what follows, the operations that are more severe come later in the order. On another note, MFA should be placed in the last "authenticated-session" step, not wrapped around an insecure SQL query.

5.1 Priority Plan

Priority	Proposed Work	Reason
P0 - Immediate	Fix both SQL injection points; migrate plaintext staff passwords to hashes; remove credential fields from reports.	Stops authentication bypass and direct credential exposure.
P1 - High	Introduce server-side RBAC; redesign donation-pass access; add MFA to staff authentication.	Reduces unauthorized access and impact of compromised credentials.
P2 - Medium	Remove unsafe rendering; add CSRF protection; harden sessions/cookies; disable debug and genericise errors.	Protects browser/session/request integrity and reduces information disclosure.
P3 - Continuous	Update vulnerable dependencies; add pytest security regression tests; repeat Semgrep/Bandit/ZAP/sqlmap/Nikto checks.	Prevents regression and supports secure maintenance over time.

Table 3: Proposed remediation priority for Part 2

5.2 Proposed Secure Development Pipeline

The proposed pipeline begins with security requirements and a threat model, followed by secure coding, static checks, automated tests, dependency auditing and controlled dynamic validation. STRIDE is suitable for the implementation report because it gives a structured way to

consider spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege around the system's trust boundaries. The detailed threat model itself is intentionally deferred to Part 2 because the assignment places threat-modelling implementation under the implementation component.

- Requirements/design gate: define authentication, authorization, data minimisation, error-handling and session requirements before code changes.
- Secure coding gate: parameterized SQL, password hashing, safe output encoding, CSRF protection and environment-based secrets.
- Static gate: run Semgrep and Bandit on application code and review findings rather than accepting/suppressing them automatically.
- Dependency gate: run pip-audit against the requirements file and evaluate upgrade paths for vulnerable packages.
- Regression gate: run pytest security tests that encode expected behaviour for injection, RBAC, MFA, CSRF, XSS, session and error controls.
- Dynamic gate: proxy functional flows through ZAP; re-run targeted sqlmap detection against the previously vulnerable parameters; use Nikto to review headers/server exposure.
- Release evidence: Keep screenshots, tool summaries and before/after explanations for Part 2 (with sensitive tokens, cookies, and MFA secrets redacted).

5.3 Proposed Acceptance Criteria for Part 2

The implementation should be deemed successful only if controls alter the structure of the code and its observable behavior. Parameterized queries must be visible in source code and original injection tests must not affect SQL. It is no longer acceptable to have passwords in a plain text field. If they request the URL, they should not be allowed to have functions that only managers have. Access to donation-pass should not be authorized with the old BDP-{id} pattern. All script strings, stored strings, should be displayed as text. Any POST request that doesn't contain a CSRF token must be denied. Session cookies should have appropriate attributes, debug should be disabled by default, and error pages should not show SQL or stack traces. Last but not least, dependency and regression-test results should be noted. These are the criteria that clearly connect the Part 1 proposal to the Part 2 implementation report.

6.0 Conclusion

Proposal shows that it's not a single patch for Blood Donation System, but multiple layer improvements are needed. There are two separate SQL injection vulnerabilities that pose an immediate threat, especially since the login vulnerability can be used to log in without being authenticated. Then the content of plaintext staff credentials and an overly accessible operational report compound the first breach. Predictable object references, stored XSS, lack of CSRF protection, weak session settings and unsafe debug / error behaviors allow for further avenues of possible unauthorized disclosure or abuse. The vulnerability set is not just a single common defect; it is the lack of secure database interaction, authentication, authorization and handling and configuration of the browser.

The proposed remediation includes parameterized queries, adaptive password hashing, server-side role enforcement, random and authorized object references, output encoding, framework CSRF protection, secure session configuration and safe error handling – all of which are solution elements of root cause remediation. To limit the impact in the event of the introduction of a further defect, defense in depth approaches like TOTP MFA, least-privilege database access, data minimization, CSP/security headers, logging and regression testing are suggested. This approach is based on NIST SSDF, as well as the current OWASP guidance.

The toolset is designed to be as varied as possible. Semgrep and Bandit offer static visibility, ZAP offers HTTP context, sqlmap verifies injections, pip-audit can check dependency risk, pytest can enable repeatable security regression testing, and Nikto can find opportunities for server/header hardening. The initial evidence gathered from the vulnerable version is enough to warrant the use of these tools and remediation plan. The next report can then concentrate on implementation, STRIDE threat modelling, changes to the code to make it secure and test results after remediation without duplicating the work of the proposal baseline.

Approximate word count: 5,375 words

7.0 References

- Abdulghaffar, K., Elmrabit, N., & Yousefi, M. (2023). Enhancing web application security through automated penetration testing with multiple vulnerability scanners. *Computers*, 12(11), 235. <https://doi.org/10.3390/computers12110235>
- Altulaihan, E. A., Alismail, A., & Frikha, M. (2023). A survey on web application penetration testing. *Electronics*, 12(5), 1229. <https://doi.org/10.3390/electronics12051229>
- Gandikota, P. S. S. K., Valluri, D., Mundru, S. B., Yanala, G. K., & Sushaini, S. (2023). Web application security through comprehensive vulnerability assessment. *Procedia Computer Science*, 230, 168-182. <https://doi.org/10.1016/j.procs.2023.12.072>
- MITRE. (2026a). CWE-89: Improper neutralization of special elements used in an SQL command (SQL injection). Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/89.html>
- MITRE. (2026b). CWE-79: Improper neutralization of input during web page generation (cross-site scripting). Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/79.html>
- National Institute of Standards and Technology. (2008). Technical guide to information security testing and assessment (NIST SP 800-115). <https://doi.org/10.6028/NIST.SP.800-115>
- National Institute of Standards and Technology. (2022). Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities (NIST SP 800-218). <https://doi.org/10.6028/NIST.SP.800-218>
- National Institute of Standards and Technology. (2025). Digital identity guidelines: Authentication and authenticator management (NIST SP 800-63B-4). <https://csrc.nist.gov/pubs/sp/800/63/B/4/final>
- OWASP Foundation. (2025a). OWASP Top 10:2025. <https://owasp.org/Top10/2025/>
- OWASP Foundation. (2025b). A01:2025 - Broken access control. OWASP Top 10:2025. https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/
- OWASP Foundation. (2025c). A02:2025 - Security misconfiguration. OWASP Top 10:2025. https://owasp.org/Top10/2025/A02_2025-Security_Misconfiguration/
- OWASP Foundation. (2025d). A03:2025 - Software supply chain failures. OWASP Top 10:2025. https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/

OWASP Foundation. (n.d.-a). SQL injection prevention cheat sheet. OWASP Cheat Sheet Series.

Retrieved August 21, 2026, from

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-b). Password storage cheat sheet. OWASP Cheat Sheet Series. Retrieved

August 21, 2026, from

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Foundation. (n.d.-c). Cross site scripting prevention cheat sheet. OWASP Cheat Sheet Series.

Retrieved August 21, 2026, from

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-d). Cross-site request forgery prevention cheat sheet. OWASP Cheat Sheet

Series. Retrieved August 21, 2026, from

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-e). Session management cheat sheet. OWASP Cheat Sheet Series. Retrieved

August 21, 2026, from

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

OWASP Foundation. (n.d.-f). Insecure direct object reference prevention cheat sheet. OWASP Cheat

Sheet Series. Retrieved August 21, 2026, from

https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-g). Multifactor authentication cheat sheet. OWASP Cheat Sheet Series.

Retrieved August 21, 2026, from

https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html

OWASP Foundation. (n.d.-h). Error handling cheat sheet. OWASP Cheat Sheet Series. Retrieved August

21, 2026, from

https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

OWASP Foundation. (n.d.-i). Authentication cheat sheet. OWASP Cheat Sheet Series. Retrieved August 21, 2026, from

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Python Packaging Authority. (n.d.). pip-audit. GitHub. Retrieved August 21, 2026, from

<https://github.com/pypa/pip-audit>

pytest developers. (n.d.). pytest documentation. Retrieved August 21, 2026, from

<https://docs.pytest.org/en/stable/>

PyCQA. (n.d.-a). Bandit documentation. Retrieved August 21, 2026, from

<https://bandit.readthedocs.io/en/latest/>

PyCQA. (n.d.-b). Bandit report formatters. Retrieved August 21, 2026, from

<https://bandit.readthedocs.io/en/latest/formatters/>

Semgrep. (n.d.). Local and CLI scans. Semgrep documentation. Retrieved August 21, 2026, from

<https://semgrep.dev/docs/category/local-and-cli-scans>

sqlmap project. (2026). Usage. sqlmap Wiki. Retrieved August 21, 2026, from

<https://github.com/sqlmapproject/sqlmap/wiki/usage>

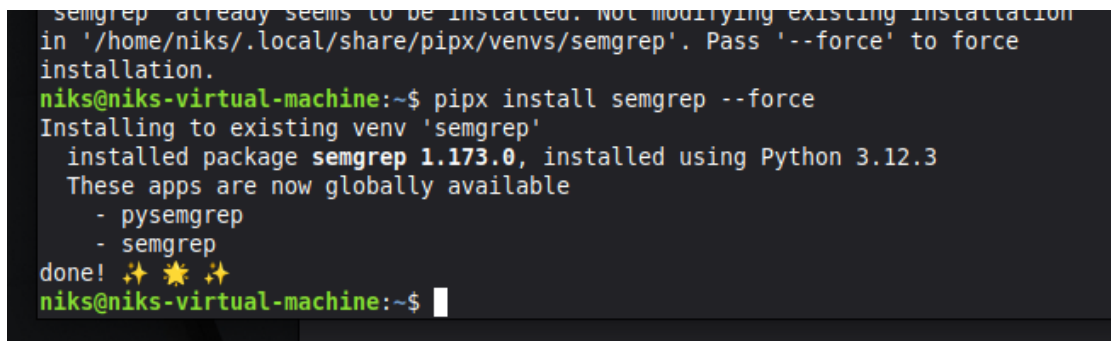
Sullo, C., & Lodge, D. (2026). Nikto vulnerability scanner manual. Nikto project. Retrieved August 21, 2026, from <https://github.com/sullo/nikto/blob/main/documentation/manpage.xml>

ZAP by Checkmarx. (n.d.). ZAP API documentation and features. Retrieved August 21, 2026, from

<https://www.zaproxy.org/docs/api/>

Appendix A Supplementary Proposal Evidence

The figures in this Appendix are supplemental setup/ vulnerable baseline numbers provided for the proposal.

A terminal window screenshot showing the installation of semgrep. The prompt is 'niks@niks-virtual-machine:~\$'. The command 'pipx install semgrep --force' is entered. The output shows that semgrep is already installed but is being updated to version 1.173.0. The terminal text is as follows:

```
semgrep already seems to be installed. Not modifying existing installation
in '/home/niks/.local/share/pipx/venvs/semgrep'. Pass '--force' to force
installation.
niks@niks-virtual-machine:~$ pipx install semgrep --force
Installing to existing venv 'semgrep'
  installed package semgrep 1.173.0, installed using Python 3.12.3
  These apps are now globally available
    - pysemgrep
    - semgrep
done! ✨ 🌟 ✨
niks@niks-virtual-machine:~$
```

Appendix Figure A1: Semgrep installation evidence on the assessment workstation

```
niks@Hemanth-virtual-machine:/opt/blood-donation-system$ cd /opt/blood-donation-system
mkdir -p evidence

semgrep --config=p/python --config=p/security-audit . | tee evidence/semgrep_old.txt
```

Scan Status

Scanning 1503 files tracked by git with 346 Code rules:

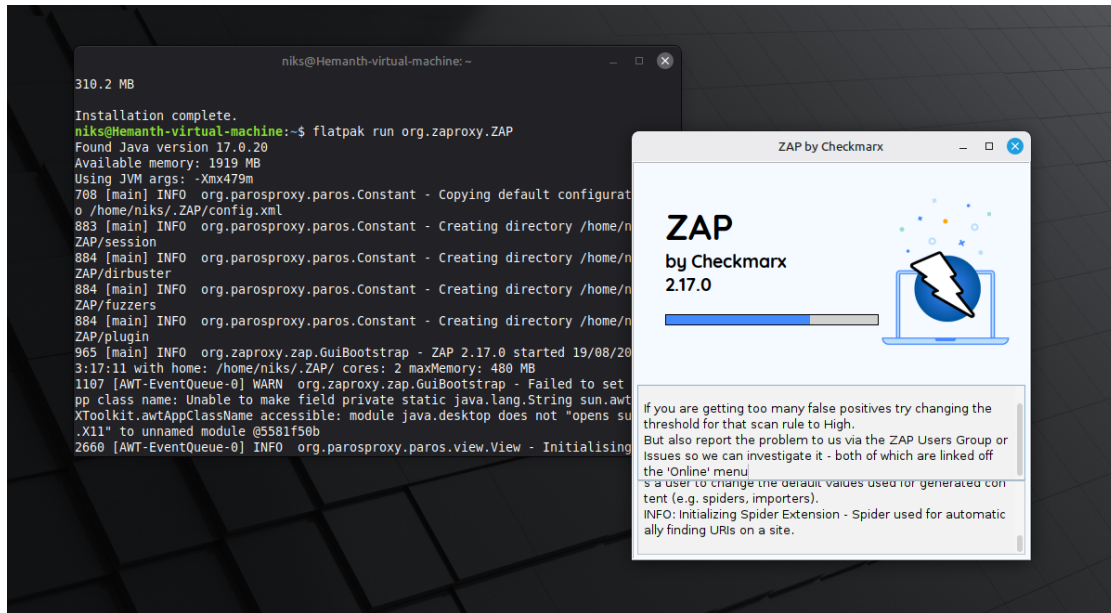
Language	Rules	Files	Origin	Rules
<multilang>	9	757	Community	346
python	198	634		
js	20	3		
c	9	1		
json	1	1		

100% 0:01:02

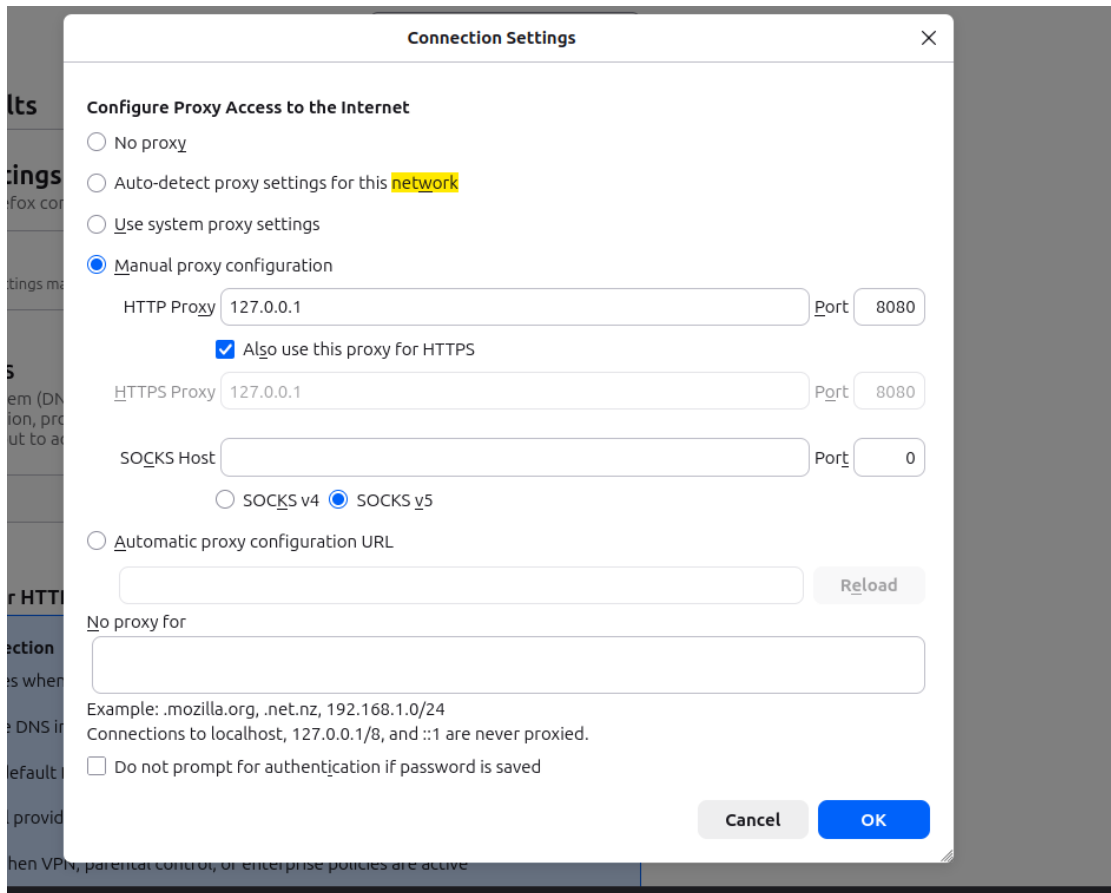
Warning: 1 timeout error(s) in venv/lib/python3.12/site-packages/idna/uts46data.py when running the following rules: [python.boto3.security.hardcoded-token.hardcoded-token]

Warning: 1 timeout error(s) in venv/lib/python3.12/site-packages/pip/_vendor/idna/uts46data.py when running the following rules: [python.boto3.security.hardcoded-token.hardcoded-token]

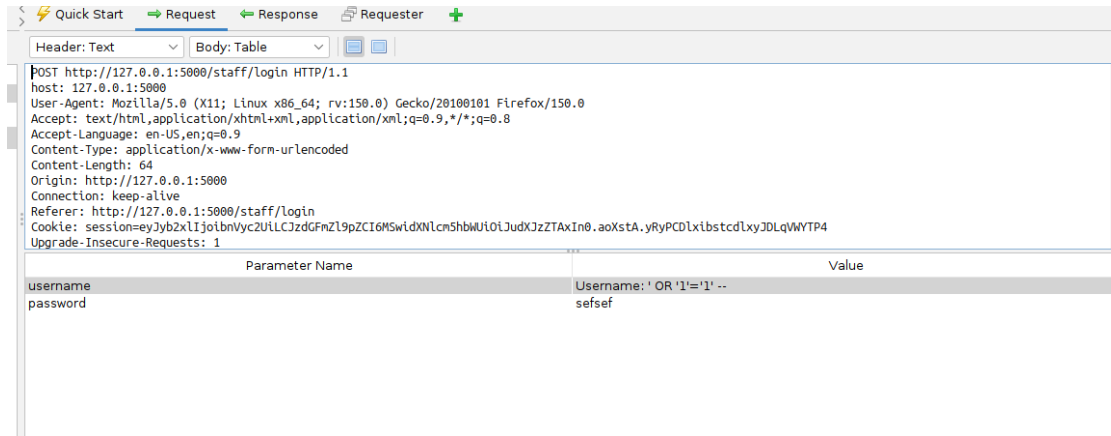
Appendix Figure A2: Semgrep scan running against the original project



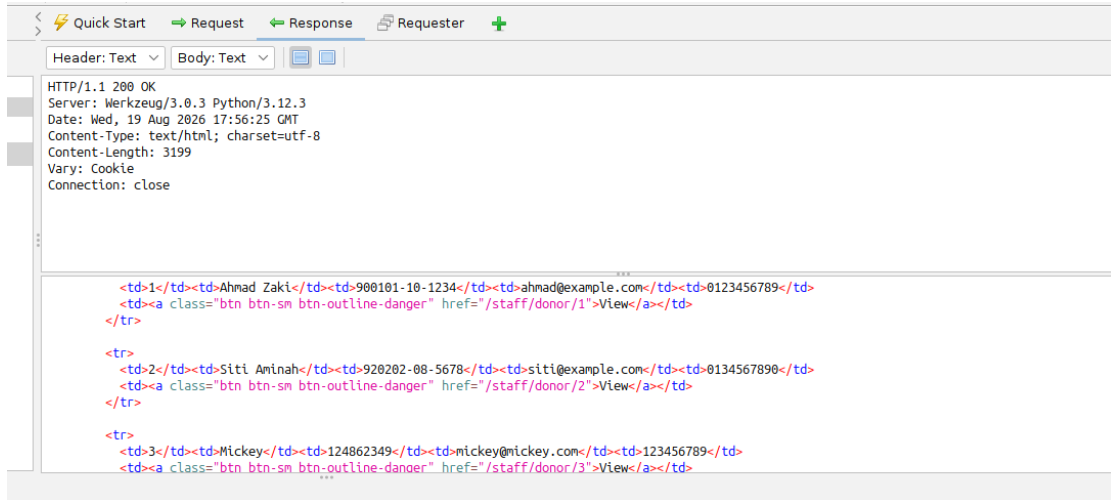
Appendix Figure A3: OWASP ZAP installation evidence



Appendix Figure A4: Firefox proxy configuration used to route local traffic through ZAP



Appendix Figure A5: Additional captured staff-login SQL injection request evidence



Appendix Figure A6: ZAP response evidence for unsafe donor-search behaviour

```
--flush-session
[+] https://sqlmap.org
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program
[*] starting @ 10:55:43 /2026-08-26/

custom injection marker ('*') found in POST body. Do you want to process it? [Y/n/q] Y
[10:55:43] [INFO] testing connection to the target URL
[10:55:43] [INFO] checking if the target is protected by some kind of WAF/IPS
[10:55:43] [INFO] testing if the target URL content is stable
[10:55:44] [INFO] target URL content is stable
[10:55:44] [INFO] testing if (custom) POST parameter '#1' is dynamic
[10:55:44] [WARNING] (custom) POST parameter '#1' does not appear to be dynamic
[10:55:44] [WARNING] heuristic (basic) test shows that (custom) POST parameter '#1' might not be injectable
[10:55:44] [INFO] testing for SQL injection on (custom) POST parameter '#1'
[10:55:44] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[10:55:44] [WARNING] reflective value(s) found and filtering out
[10:55:45] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[10:55:45] [INFO] (custom) POST parameter '#1' appears to be 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)' injectable (with --code=200)
[10:55:45] [INFO] testing 'Generic inline queries'
[10:55:45] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[10:55:45] [INFO] (custom) POST parameter '#1' is 'PostgreSQL AND error-based - WHERE or HAVING clause' injectable
for the remaining tests, do you want to include all tests for 'PostgreSQL' extending provided level (2) and risk (1) values? [Y/n] Y
[10:55:45] [INFO] testing 'PostgreSQL inline queries'
[10:55:45] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
```

Appendix Figure A7: sqlmap detection process against the original login request

```
these apps are now globally available
- pip-audit
done! ✨ ✨ ✨ ✨
niks@Hemanth-virtual-machine:/opt/blood-donation-system$ pip-audit --version
pip-audit 2.10.1
niks@Hemanth-virtual-machine:/opt/blood-donation-system$
```

Appendix Figure A8: pip-audit version confirmation in the original environment

```
niks@Hemanth-virtual-machine:~$ nikto -Version
-----
Nikto Versions
-----
File                               Version      Last Mod
-----
Nikto main                         2.1.5
LibWhisker                         2.5
CHANGES.txt
db_404_strings                     2.003
db_content_search                  2.000
2012-07-04                         1.0
db_embedded                        2.004
db_favicon                         2.010
db_headers                         2.008
db_httptoptions                    2.002
db_multiple_index                  2.005
db_outdated                        2.017
db_parked_strings                  2.000
db_realms                          2.002
db_server_msgs                     2.006
db_subdomains                      2.006
db_tests                           2.019
db_variables                       2.004
nikto_apache_expect_xss.plugin     2.04
nikto_apacheusers.plugin           2.06
nikto_auth.plugin                  2.04
nikto_cgi.plugin                   2.06      2008-05-06
nikto_clientaccesspolicy.plugin    1.00
nikto_content_search.plugin        2.05
nikto_cookies.plugin               2.03
nikto_core.plugin                  2.1.5
nikto_dictionary_attack.plugin     2.04
nikto_embedded.plugin              2.07
nikto_favicon.plugin               2.09
nikto_fileops.plugin               1.00
nikto_headers.plugin               2.10
nikto_httptoptions.plugin           2.10
nikto_msgs.plugin                  2.07
nikto_multiple_index.plugin        2.03
nikto_outdated.plugin              2.09
nikto_parked.plugin                2.00
nikto_paths.plugin                 2.00
nikto_put_del_test.plugin           2.04
nikto_report_csv.plugin            2.06      2008-11-11
nikto_report_html.plugin            2.05      2009-07-20
```

Appendix Figure A9: Nikto installation/version evidence